MIE1517 More Sample Questions

Part 6: Generative Adversarial Networks

"Training": (whether you "know" the material)

1. List three generative models we discussed in class. How are they similar? How are they different?
   Answer: Autoencoder, RNN for text generation, GAN. They are used to solve unsupervised problems. The architectures of the models are very different. Autoencoder has an encoder and decoder and it is often used to produce low dimensional representations of the original dataset. Generative RNN is commonly used for text generation. This model learns based on the context and the ordering of text. A GAN has a generator and discriminator. The generator and discriminator try to outperform each other. The generator tries to produce fake data that fools discriminator while the discriminator tries to distinguish between the real and fake data.
2. Why do autoencoders produce blurry images?
   Answer: Because of the MSE loss function. The background of the images can be really different, so the only way to minimize loss is to produce an average of pixel intensities. That's why the edges look blurry and the background looks grey.
3. What does the generator of a GAN optimize? What does the discriminator of a GAN optimize?
   Answer: The generator maximizes the probability of the discriminator labeling fake generated images as being real. The discriminator maximizes the probability of the discriminator labeling real images as real and labeling generated images as fake.
4. Why did we need two different torch.optim optimizers to optimize the generator and discriminator?
   Answer: Discriminator is trying to maximize P(D predicting real when image as real) and P(D predicting false image as being fake)

   Generator is trying to minimize P(D predicting false when fake image generated by G)
5. What is batch normalization? How is it similar to normalizing the input? How is it different?
   Answer: If each input neuron is roughly of the same scale, then we can use the same method to initialize all of our weights and biases.

   Batch normalization normalizes the hidden activation to have mean 0 and standard deviation of 1 (or some other value). At train time, it performs the normalization across each mini-batch, but also keep track of the means and standard deviations of incoming activations. At test time, we use the mean and standard deviations learned during training to normalize the test input.
6. Why is training a GAN difficult?
   Answer: One difficulty is that a training curve is no longer as helpful as it was for a supervised learning problem! The generator and discriminator losses tend to bounce up and down, since both the generator

and discriminator are changing over time. Tuning hyperparameters is also much more difficult, because we don't have the training curve to guide us.

7. What is the difference between a targeted and untargeted adversarial attack?

   Answers: In a targeted attack, we want the model to misclassify perturbed images into a class of our choosing. In an untargeted adversarial attack, we want the model to misclassify perturbed images into any class.

8. What is the difference between a white-box and black-box adversarial attack?

   Answers: White Box: Architecture and weights of network are know to optimize perturbation.

   Black Box: Architecture and weights of network are not known. Substitute model mimicking target model with differentiable function

Generalization: (whether you can apply the material)

1. Why is it that a GAN is more likely to suffer from mode collapse than an autoencoder?

   Answer: Assume MNIST dataset. Mode collapse refers to the phenomenon that a GAN model only outputs a small number of digit types. A GAN is more likely to suffer from mode collapse because a generator can optimize P(D correctly identifies image generated by G) by learning to generate one type of input (e.g. One digit) really well, and not learning how to generate any other digits at all.

2. Would our RNN text generation model also suffer from mode collapse? Why or why not?

   Answer: No. We can create different patterns of output by adjusting the temperature. The output also depends on the training data. The RNN text generation model is unlikely to generate the same text all the time since the text is sampled.

3. Why is the balance between generator and discriminator important? Which one is more likely to become too powerful and cause an issue?

   Answers: The discriminator is more likely to become too powerful because supervised learning is an easier task in machine learning. If the discriminator is too good, then the generator will not learn. Because we are using the discriminator as a "loss function" for the generator. If the discriminator is too good, small changes in the generator weights won't change the discriminator output. If small changes in generator weights make no difference, then we can't incrementally improve the generator.

Part 7: Recurrent Neural Networks

"Training": (whether you "know" the material)

1. What is the purpose of (word2vec, GloVe)?

   Answer: Provide a lower dimensional representation of words, while at the same time it tries to capture the meaning of the words.

2. Describe what it means to use a one-hot embedding representation of a categorical feature.

Answer: One-hot encoding is a way to convert categorical features into numerical ones. Each feature is represented by a unique vector and one and only one entry in the vector is a "1"

Example:

Fall -> [1 0 0]

Winter -> [0 1 0]

Summer -> [0 0 1]

3. What is the advantage of nn.GRU and nn.LSTM over nn.RNN?

   Answer: nn.LSTM and nn.GRU are better at perpetuating long-term dependencies. nn.LSTM keeps track of both a hidden state and a cell state and has three gates (input, output and forget gates) while nn.GRU only has two gates (reset and update gates).

4. What is the advantage of nn.GRU over nn.LSTM?

   Answer: nn.GRU uses less memory. It controls the flow of information like the LSTM unit, but without having to use a memory unit (cell unit). It just exposes the full hidden content without any control. nn.GRU is more efficient (less complex structure) and gives the comparable performance

5. Explain why padding is sometimes necessary when training an RNN.

   Answer: We need to make sure that all the inputs inside a mini batch have the same sequence length

Generalization: (whether you can apply the material)

1. How can you use autoencoders for data augmentations?

   Answer: One example is to take two images in and get their low dimensional representation and do linear interpolation.

2. Consider embedding a tweet by summing the GloVe embeddings of the words that appear in the tweet. Suppose that you would like to measure "how similar" two tweets are, using this tweet embedding. Would you use the cosine similarity or the Euclidean distance?

   Answer: Cosine Similarity. The reason is to remove the effect of the tweet length on similarity.

3. Character-level RNNs are RNNs that takes in a text input one character (one letter) at a time. What are advantages of a character-level RNN (compared to a word-level RNN?)? What are the disadvantages of a character level RNN?

   Answer:

   Advantages:

   1. Can create new words and learn from data that are poorly spelled (ex. tweets)
   2. Takes less memory space since there are more words than characters

   Disadvantages:

   1. Difficult to create coherent text messages. The model needs to learn how to spell properly and how to use correct grammar.
   2. The model will take longer to get good performance

Part 8: Generative RNNs

"Training": (whether you "know" the material)

1. What is the "temperature"?
   Answer: The temperature modifies the randomness of the output probability distribution. With a high temperature, the model will produce more random output. With low temperature, the model will be more likely to produce outputs that have a higher probability.

2. When generating text using a trained RNN model, do we obtain cleaner samples using a high temperature or a low temperature?
   Answer: We obtain cleaner samples using a low temperature. Because with a low temperature, the model is more likely to pick the character that has a higher probability.

3. What is "teaching-forcing"? Is it used during training or evaluation?
   Answer: It is a method used to train generative RNN. During training, it updates the hidden states with the ground truth token regardless of the prediction from the previous step. It is used during training.

4. What loss function do we use when training an RNN model to generate text? Why?
   Answer: Cross-entropy loss. Because each word represents a class and for each iteration, we need to compare the word with the ground truth. This is clearly a multi-class classification problem.

Generalization: (whether you can apply the material)

1. We did not set aside a "test set" for our generative RNN model. Why?
   Answer: Because this is an unsupervised problem. The data does not have labels.
2. Is text generation a supervised or unsupervised learning problem?
   Answer: An unsupervised problem. Because there is no right or wrong answer to this problem.
3. During training, we use the ground-truth sequence as input to the RNN. However, during test time, we used the generated sequences as input. Can the disconnect between what is done during training and what is done for testing be a problem? Why or why not?
   Answers: No. During training we want the model to learn the pattern in the training set because we need some kind of ground truth. (even though this is an unsupervised problem) During testing, we want this model to generate a random pattern, so we will feed its generative output to the next input.

Part 9: Transformers

"Training": (whether you "know" the material)

1. What is an attention mechanism and why how does it help?

Answer: An attention mechanism is a technique initially used in Encoder–Decoder models to give the decoder more direct access to the input sequence, allowing it to deal with longer input sequences. At each decoder time step, the current decoder's state and the full output of the encoder are processed by an alignment model that outputs an alignment score for each input time step. This score indicates which part of the input is most relevant to the current decoder time step. The weighted sum of the encoder output (weighted by their alignment score) is then fed to the decoder, which produces the next decoder state and the output for this time step. The main benefit of using an attention mechanism is the fact that the Encoder–Decoder model can successfully process longer input sequences.

2. What is a Transformer and how is it different from an RNN?
   Answer: The main difference is that the Transformer takes in the entire sequence and can operate in parallel unlike RNNs that examine the input sequentially.

3. What is the most important layer in the Transformer architecture and what is its purpose?
   Answer: The most important layer in the Transformer architecture is the Multi-Head Attention layer (the original Transformer architecture contains 18 of them, including 6 Masked Multi-Head Attention layers). It is at the core of language models such as BERT and GPT-2. Its purpose is to allow the model to identify which words are most aligned with each other, and then improve each word's representation using these contextual clues.

4. When would you need to use sampled softmax?
   Answer: Sampled softmax is used when training a classification model when there are many classes (e.g., thousands). It computes an approximation of the cross-entropy loss based on the logit predicted by the model for the correct class, and the predicted logits for a sample of incorrect words. This speeds up training considerably compared to computing the softmax over all logits and then estimating the cross-entropy loss. After training, the model can be used normally, using the regular softmax function to compute all the class probabilities based on all the logits.

Generalization: (whether you can apply the material)

1. How can the inclusion of attention mechanism help with debugging the model?
   Answers: The alignment scores makes the model easier to debug and interpret: for example, if the model makes a mistake, you can look at which part of the input it was paying attention to, and this can help diagnose the issue.

2. Which of the following are T/F:
   a. The dimensionality of queries has to match the dimensionality of the keys? Answer: True
   b. The number of keys must match the number of values? Answer: True

c. The number of keys have to match the number of queries? Answer: False

3. What are some problems with recurrent architectures?
   Answer: Not parallelizable across instances, cannot model long-term dependences, vanishing/exploding gradients.

Part 10: Ethics and AI

"Training": (whether you "know" the material)

1. How can an unrepresentative training data pose fairness issues?
   Answer: Machine learning models learn from unrepresentative training data. The models' prediction they make will be unfair because the data used are biased.
2. If we removed the sensitive features of a model, will our model achieve have equalized odds?
   Answer: No because removing sensitive features of a model will cause the model to learn differently creating different false-negative and false-positive rate thus the model does not achieve equalized odds.
3. What is one problem with using demographic parity as a measure of fairness?
   Answer: Fairness is measured at a group level. Model can hire qualified people from one group, and random people from the other (because there are no people actually qualified in the other group)
4. What is one problem with using equalized odds as a measure of fairness?
   Answer: False positive and false negative have different impacts. It does not help close the gap between the two groups
5. What evidence is there that word2vec embeddings can be problematic?
   Answer: Biases in word2vec embeddings. Models learn bias in the training data.
   Doctor−man+woman = nurse

   Word2vec is problematic when facing unknown or out-of-vocabulary words, since it assigns random vector to unknown diction which might be far away from ideal

Generalization: (whether you can apply the material)

1. How can the gesture recognition model we trained in our assignment pose a fairness issue?
   Answer: Might provide higher accuracy for certain gender if there are more data provided by that gender
2. How can the spam detection model we trained in assignment pose a fairness issue?
   Answer: If the data trained comes from one demographic then the model may be better able to sort "spam" and "non-spam" for the demographic it was trained on.
3. Show, using example (hypothetical) data, of how equalized odds and demographic parity can be contradictory goals.
   Answer: A luxury restaurant would like to establish a promotion to loyal clients. Clients are consisted of two groups---- a subsets of affluent old moneys and a subset of less affluent blue collars. This situation is

fine with demographic parity solely, as long as the same fraction of people in both class are selected. However, fulfilling the same acceptance rate among two groups and the same proportion of qualified people among two different groups at the same time may be a contradiction. The affluent class is certain to have more qualified candidates than the blue-collar class. It is never the restaurant's interest to lose any potential customers or disappoint existing customers. For example, imagine affluent class has 100 applicants and 58 of them are qualified while blue collars also have 100 applicants but only 2 of them are qualified As a result, the restaurant will offer 29 promotion to affluent class and 1will be offered to the blue collar class. This contradicts with the demographic parity policy. Alternatively, the restaurant picks a number of qualified customers from the affluent class and random people from the blue-collar class. There are definitely less qualified candidates in the selected portion of the blue-collar class than the affluent class.